

Optimización de Código y Medición de Tiempos

Introducción

El objetivo de este trabajo autónomo fue el análisis y la optimización de código en Python que se encarga de identificar números primos en el rango 1 a 100 000. El código original utilizaba un método deficiente, basado en recorrer todos los divisores posibles desde 2 hasta n , lo cual generaba un tiempo de ejecución elevado y un consumo significativo de recursos computacionales.

Además, se aplicaron herramientas de medición de tiempos (`time`) y análisis de rendimiento (`cProfile`) para evaluar el impacto de las mejoras implementadas. Finalmente, se generaron gráficos comparativos utilizando `Matplotlib`.

Optimización Aplicada

En matemáticas, para determinar si un número es primo solo es necesario verificar divisores hasta la raíz cuadrada del mismo.

Se reemplazó la construcción manual de listas mediante `.append()` por comprensiones de lista, lo que mejora la eficiencia y limpieza del código.

Se implementó una versión adicional que emplea `numpy.arange()` para generar el rango de números de forma más eficiente.

Resultados Obtenidos

Versión del código Tiempo total Función más costosa

Original **9.690 s** `es_primo`

Optimizado **0.101 s** `es_primo_opt`

Análisis con cProfile

Código original 109,596 llamadas totales La función `es_primo` tardó 9.670 s Representó el 99.8% del tiempo total

Código optimizado 200,003 llamadas totales (por uso de `math.sqrt`) La función `es_primo_opt` tardó 0.080 s `math.sqrt` tardó 0.008 s El tiempo total fue 0.101 s

La reducción del rango de búsqueda fue la mejora más significativa, eliminando miles de iteraciones innecesarias por cada número evaluado.

Conclusiones

La optimización del algoritmo permitió una reducción drástica en el tiempo de ejecución, pasando de 9.69 s a 0.10 s, demostrando la importancia de aplicar principios matemáticos y buenas prácticas en programación.

El uso de list comprehensions y NumPy contribuyó a un código más limpio, legible y eficiente.

Herramientas como `cProfile` son esenciales para identificar cuellos de botella y tomar decisiones fundamentadas de optimización.

Es fundamental en la Ciencia de Datos, donde la eficiencia de código puede marcar la diferencia en el tiempo de procesamiento de proyectos